Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 06

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Identificar la funcionalidad del motor de plantillas de Django (herencia, filtros, inclusión) dentro de una aplicación empresarial.**
2. **Refactorizar Views y Templates ya existentes para que reutilicen una estructura común mediante herencia de plantillas (extends / block).**
3. **Aplicar la sintaxis avanzada de plantillas (filtros, comentarios, include) y verificar el auto-escape como mecanismo de seguridad.**

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la Semana 6 (motor de plantillas de Django: herencia, filtros, include, seguridad) antes del desarrollo del laboratorio.

III. Normas empleadas

- No aplica

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.
- La aplicación con CRUD completo (URLs, Views y Templates) desarrollada desde la Semana 3, ampliada con relaciones en la Semana 4 y registrada en el Django Admin en la Semana 5.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código (antes y después del cambio) en el Template correspondiente, además de la página renderizada en el navegador.
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

VII. Procedimiento

Desde la Semana 3, cada estudiante ya cuenta con URLs, Views y Templates propios que implementan CRUD completo, ampliados en la Semana 4 con relaciones y expuestos también en el Django Admin en la Semana 5. En esta sesión no se crean Views ni Templates nuevos desde cero: se refactorizan los que ya existen para que reutilicen una estructura común y apliquen buenas prácticas de plantillas.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante retoma la aplicación de la Semana 4/5 y migra sus Templates existentes a herencia con base.html, agrega filtros, comentarios e include. En la segunda parte, deberá hacer lo mismo sobre los Templates de su

investigación propia (las siete entidades ya registradas en el Admin en la Semana 5), cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda que la sintaxis de plantillas no reemplaza el CRUD que ya construyó: lo hace más mantenible y seguro. El mismo Template que ya muestra sus datos pasa a heredar una estructura común, a reutilizar fragmentos con include, y a apoyarse en el auto-escape de Django como defensa contra XSS.

Cliente (navegador)

→ URL ya existente (ej. /productos/)

→ urls.py (sin cambios)

→ View ya existente (views.py)

→ Manager / QuerySet

→ Django ORM

→ SQLite

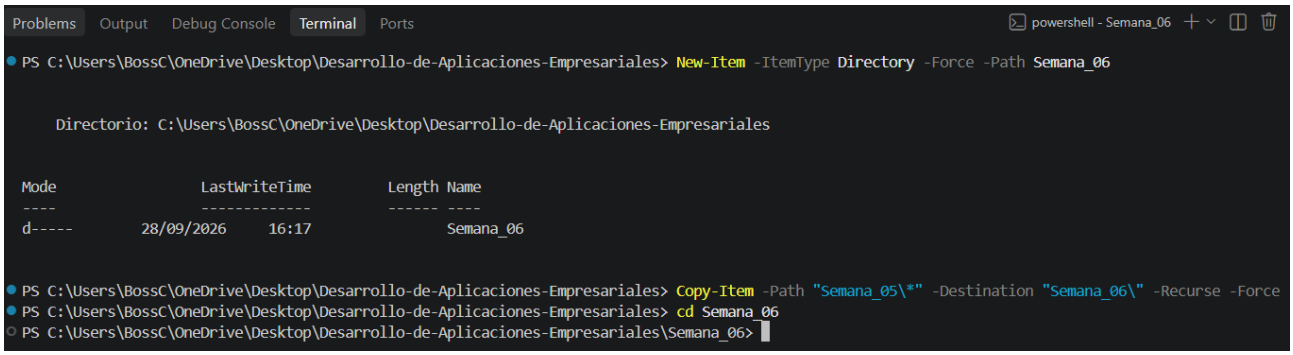
→ Template refactorizado (extends / include / filtros)

→ Response

Al finalizar el laboratorio, el estudiante deberá ser capaz de migrar un Template existente a herencia con extends y block sobre una base.html común, aplicar filtros y comentarios, reutilizar fragmentos con include, y verificar el auto-escape de Django como mecanismo de seguridad frente a XSS.

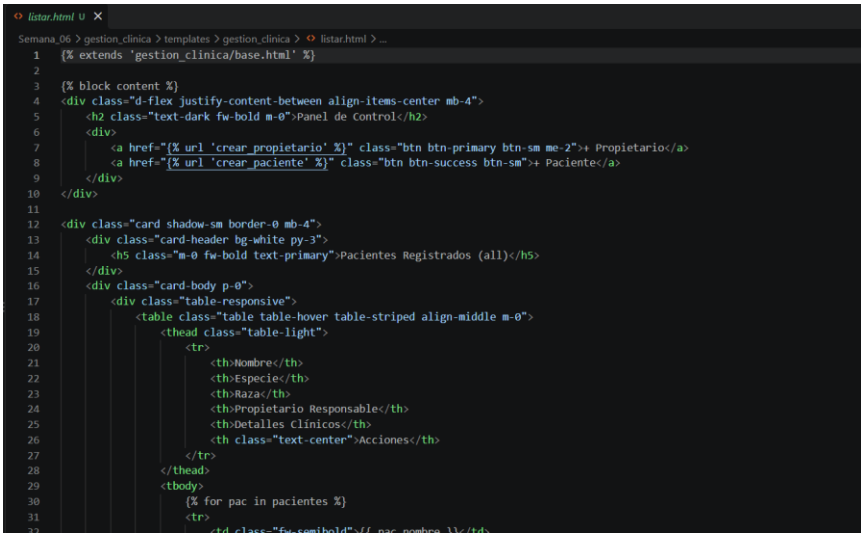
Enunciados

PARTE 1 — Refactorizar los Templates ya existentes de la aplicación de la Semana 4/5



Ejercicio 1 — Auditar tus Templates actuales

Revise los Templates que ya tiene funcionando desde la Semana 3/4 (listado, formulario, detalle) e identifique si alguno ya usa herencia (extends). Documente el estado actual con una captura de cada Template.



```
listar.html U
Semana_06 > gestion_clinica > templates > gestion_clinica > listar.html > ...
{% extends 'gestion_clinica/base.html' %}

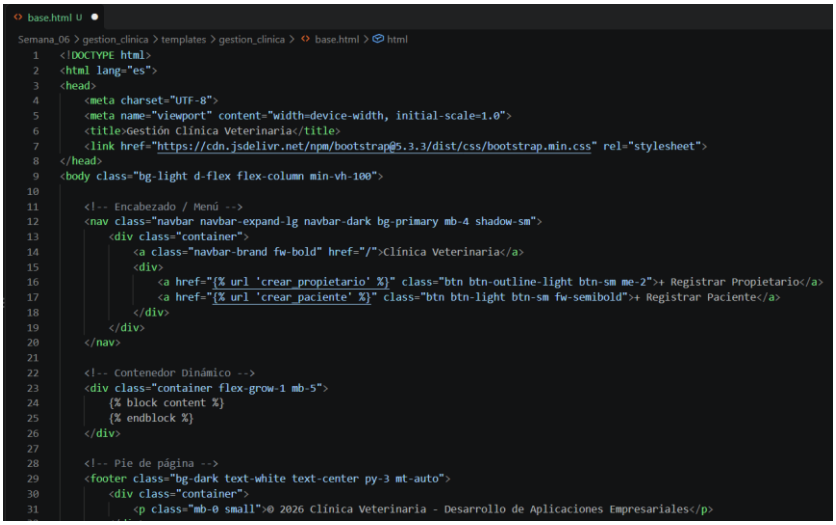
{% block content %}
<div class="d-flex justify-content-between align-items-center mb-4">
    <h2 class="text-dark fw-bold m-0">Panel de Control</h2>
    <div>
        <a href="{% url 'crear_propietario' %}" class="btn btn-primary btn-sm me-2">+ Propietario</a>
        <a href="{% url 'crear_paciente' %}" class="btn btn-success btn-sm">+ Paciente</a>
    </div>
</div>

<div class="card shadow-sm border-0 mb-4">
    <div class="card-header bg-white py-3">
        <h5 class="m-0 fw-bold text-primary">Pacientes Registrados (all)</h5>
    </div>
    <div class="card-body p-0">
        <div class="table-responsive">
            <table class="table table-hover table-striped align-middle m-0">
                <thead class="table-light">
                    <tr>
                        <th>Nombre</th>
                        <th>Especie</th>
                        <th>Raza</th>
                        <th>Propietario Responsable</th>
                        <th>Detalles Clínicos</th>
                        <th class="text-center">Acciones</th>
                    </tr>
                </thead>
                <tbody>
                    {% for pac in pacientes %}
                    <tr>
                        <td class="fw-semibold">{{ pac.nombre }}</td>
```

Revisé los templates que construí en las semanas anteriores para el CRUD de mi clínica veterinaria (como los listados y formularios). Noté que en todos estaba repitiendo la misma estructura de HTML básico (head, body, links a Bootstrap y la barra de navegación). Hasta ahora, ninguno de estos archivos utilizaba la herencia con la etiqueta extends, por lo que el código era muy redundante y difícil de mantener si quería cambiar un estilo global.

Ejercicio 2 — Crear base.html

Si aún no existe, cree base.html con la estructura común del sitio (encabezado, menú, pie de página) y un bloque {% block content %} vacío. No modifique todavía sus Views.



```
base.html U
Semana_06 > gestion_clinica > templates > gestion_clinica > base.html > html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Gestión Clínica Veterinaria</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light d-flex flex-column min-vh-100">

    <!-- Encabezado / Menú -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-primary mb-4 shadow-sm">
        <div class="container">
            <a class="navbar-brand fw-bold" href="/">Clínica Veterinaria</a>
            <div>
                <a href="{% url 'crear_propietario' %}" class="btn btn-outline-light btn-sm me-2">+ Registrar Propietario</a>
                <a href="{% url 'crear_paciente' %}" class="btn btn-light btn-sm fw-semibold">+ Registrar Paciente</a>
            </div>
        </div>
    </nav>

    <!-- Contenedor Dinámico -->
    <div class="container flex-grow-1 mb-5">
        {% block content %}
        {% endblock %}
    </div>

    <!-- Pie de página -->
    <footer class="bg-dark text-white text-center py-3 mt-auto">
        <div class="container">
            <p class="mb-0 small">© 2026 Clínica Veterinaria - Desarrollo de Aplicaciones Empresariales</p>
```

Para cumplir con este paso, creé el archivo base.html en mi carpeta de plantillas. Estructuré el documento utilizando HTML5 e integré Bootstrap para mantener un diseño profesional. Definí una estructura común con un encabezado (navbar), un pie de página estático (footer), y agregué la etiqueta {% block content %} en el centro del cuerpo. Este bloque servirá como un contenedor dinámico vacío para que las demás plantillas puedan inyectar su código específico sin necesidad de repetir la estructura principal. Aún no modifiqué ninguna View.

Ejercicio 3 — Migrar un Template existente a herencia

Modifique uno de sus Templates ya funcionando para que use {% extends "base.html" %} y {% block content %}, sin cambiar la View que lo renderiza. Verifique que la página sigue mostrando los mismos datos.

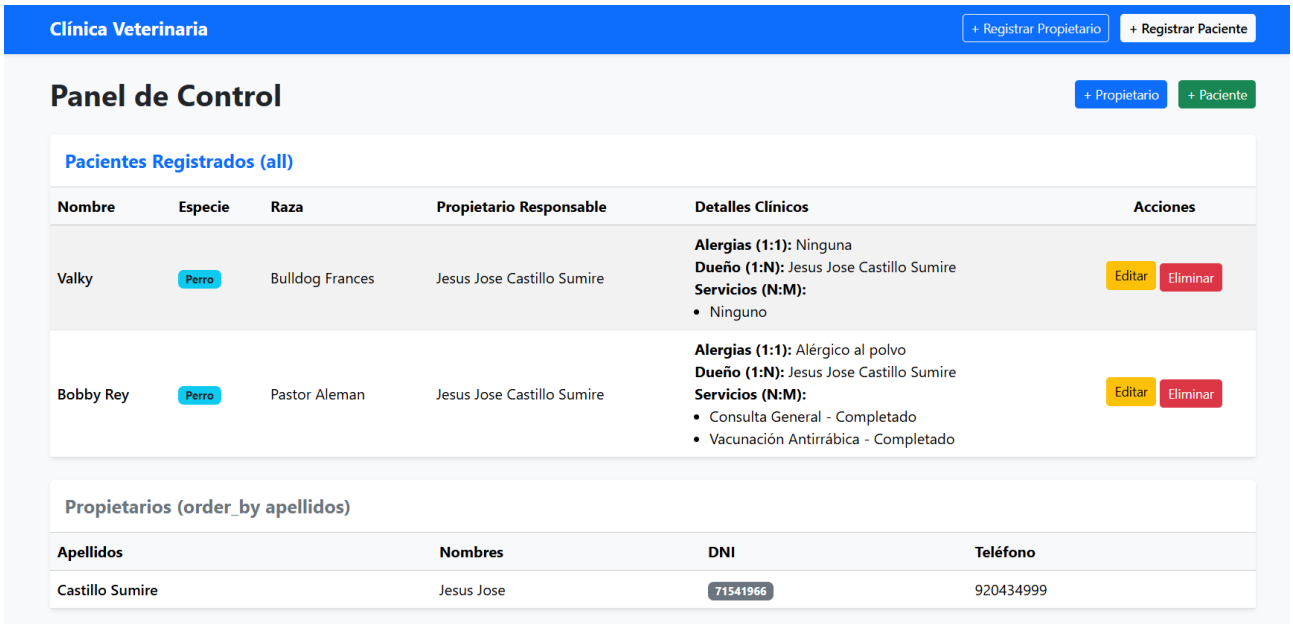
base.html U listar.html U

Semana_06 > gestion_clinica > templates > gestion_clinica > listar.html > ...

```
{% extends 'gestion_clinica/base.html' %}

{% block content %}
<div class="d-flex justify-content-between align-items-center mb-4">
    <h2 class="text-dark fw-bold m-0">Panel de Control</h2>
    <div>
        <a href="{% url 'crear_propietario' %}" class="btn btn-primary btn-sm me-2">+ Propietario</a>
        <a href="{% url 'crear_paciente' %}" class="btn btn-success btn-sm">+ Paciente</a>
    </div>
</div>

<div class="card shadow-sm border-0 mb-4">
    <div class="card-header bg-white py-3">
        <h5 class="m-0 fw-bold text-primary">Pacientes Registrados (all)</h5>
    </div>
    <div class="card-body p-0">
        <div class="table-responsive">
            <table class="table table-hover table-striped align-middle m-0">
                <thead class="table-light">
                    <tr>
                        <th>Nombre</th>
                        <th>Especie</th>
                        <th>Raza</th>
                        <th>Propietario Responsable</th>
                        <th>Detalles Clínicos</th>
                        <th class="text-center">Acciones</th>
                    </tr>
                </thead>
                <tbody>
                    {% for pac in pacientes %}
                    <tr>
                        <td class="fw-semibold">{{ pac.nombre }}</td>
```



Modificando mi plantilla listar.html (que renderiza la lista principal de pacientes), eliminé toda la estructura base de HTML (head, body, links a CSS). En la primera línea coloqué la etiqueta {% extends 'gestion_clinica/base.html' %} para heredar el diseño maestro, y envolví el código de mi tabla dentro de la etiqueta {% block content %}. Al recargar el navegador, verifiqué que la página sigue mostrando los mismos datos correctamente, demostrando que la View en Python no sufrió cambios y sigue enviando los datos al nuevo esquema de la plantilla.

Ejercicio 4 — Migrar el resto de tus Templates

Repita la migración a herencia en al menos dos Templates más (por ejemplo, el listado y el formulario), reutilizando el mismo base.html.

base.html U | listar.html U | crear_paciente.html U | crear_propietario.html U

Semana_06 > gestion_clinica > templates > gestion_clinica > crear_propietario.html > ...

```
{% extends 'gestion_clinica/base.html' %}

{% block content %}
<div class="row justify-content-center">
    <div class="col-md-8">
        <div class="card shadow-sm border-0 mt-4">
            <div class="card-header bg-primary text-white py-3">
                <h5 class="m-0 fw-bold">Registrar Nuevo Propietario</h5>
            </div>
            <div class="card-body">
                <!-- Tu formulario existente -->
                <form method="POST">
                    {% csrf_token %}
                    {{ form.as_p }}
                    <div class="d-flex justify-content-end mt-4">
                        <a href="/" class="btn btn-secondary me-2">Cancelar</a>
                        <button type="submit" class="btn btn-primary">Guardar Propietario</button>
                    </div>
                </form>
            </div>
        </div>
    </div>
</div>
{% endblock %}
```

base.html U | listar.html U | crear_paciente.html U | crear_propietario.html U

Semana_06 > gestion_clinica > templates > gestion_clinica > crear_paciente.html > ...

```
{% extends 'gestion_clinica/base.html' %}

{% block content %}
<div class="row justify-content-center">
    <div class="col-md-8">
        <div class="card shadow-sm border-0 mt-4">
            <div class="card-header bg-success text-white py-3">
                <h5 class="m-0 fw-bold">Registrar Nuevo Paciente</h5>
            </div>
            <div class="card-body">
                <!-- Tu formulario existente -->
                <form method="POST">
                    {% csrf_token %}
                    {{ form.as_p }}
                    <div class="d-flex justify-content-end mt-4">
                        <a href="/" class="btn btn-secondary me-2">Cancelar</a>
                        <button type="submit" class="btn btn-success">Guardar Paciente</button>
                    </div>
                </form>
            </div>
        </div>
    </div>
</div>
{% endblock %}
```

Continuando con la refactorización, seleccioné dos plantillas adicionales correspondientes a la creación de registros: el formulario de pacientes y el formulario de propietarios. En ambos archivos eliminé el código HTML repetitivo y utilicé las etiquetas {% extends 'gestion_clinica/base.html' %} junto con {% block content %} para heredar el esquema maestro. Al navegar por el sistema, verifiqué que los formularios de ingreso de datos se renderizan perfectamente manteniendo la barra de navegación superior y el pie de página, demostrando una reutilización de código eficiente.

Ejercicio 5 — Aplicar un filtro sobre un dato ya mostrado

En un Template que ya muestra datos de su entidad, aplique al menos un filtro (por ejemplo, date, floatformat, upper o length) sobre un campo existente.

```
32                  <td class="fw-semibold">{{ pac.nombre }}</td>
33                  <td class="fw-semibold">{{ pac.nombre|upper }}</td>
34                  <td><span class="badge bg-info text-dark">{{ pac.especie }}</span></td>
```

```
90              <td colspan="4" class="text-center text-muted py-3">No hay propietarios registrados.</td>
91              <td class="fw-bold">S/. {{ med.precio|floatformat:2 }}</td>
```

Para aplicar los filtros de plantilla solicitados, modifiqué mi archivo listar.html. Utilicé el filtro upper en la variable del nombre del paciente ({{ pac.nombre|upper }}) para estandarizar su visualización en mayúsculas en el panel de control, sin modificar el dato original en la base de datos. Además, apliqué el filtro floatformat:2 al precio de los medicamentos para garantizar que siempre se muestren con dos decimales, mejorando la presentación financiera de la tabla.

Ejercicio 6 — Documentar con comentarios

Agregue al menos un comentario {# #} en un Template, documentando una sección que pueda no ser evidente para otro desarrollador.

```
<td>
    <strong>Alergias (1:1):</strong> {{ pac.historial.alergias|default:"Ninguna" }}<br>
    <strong>Dueño (1:N):</strong> {{ pac.propietario.nombres }} {{ pac.propietario.apellidos }}<br>
    {# Iteramos sobre el related_name inverso (pacienteservicio_set) en lugar de ir directo al servicio para poder extraer atrib
    <strong>Servicios (N:M):</strong>
    <ul class="mb-0 ps-3">
```

Para documentar mi código, inserté un comentario de bloque de Django {# #} dentro de la plantilla listar.html. Elegí documentar la sección donde se extraen los servicios del paciente, explicando que iteramos sobre pacienteservicio_set (el modelo intermedio) en lugar de la tabla de servicios directa. Esto es crucial para que cualquier otro desarrollador entienda rápidamente por qué se estructuró así la consulta y cómo nos permite extraer el 'estado' de la atención médica.

Ejercicio 7 — Reutilizar con include

Identifique un fragmento que se repite entre dos de sus Templates (por ejemplo, la tarjeta de un registro en un listado) y extráigalo a un archivo aparte para insertarlo con {% include %}.

base.html U | listar.html U | _formulario.html U | crear_paciente.html U | crear_propietario.html U

Semana_06 > gestion_clinica > templates > gestion_clinica > _formulario.html > form

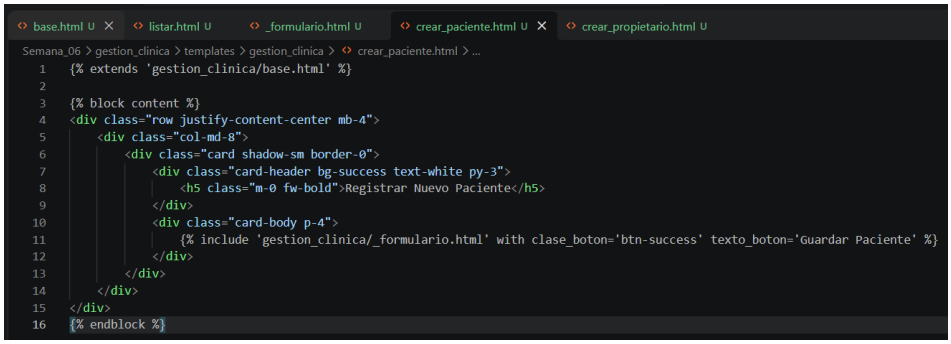
```
<form method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <div class="d-flex justify-content-end mt-4 gap-2">
        <a href="/" class="btn btn-secondary">Cancelar</a>
        <button type="submit" class="btn {{ clase_boton|default:'btn-primary' }}">{{ texto_boton|default:'Guardar' }}</button>
    </div>
</form>
```

base.html U | listar.html U | _formulario.html U | crear_paciente.html U | crear_propietario.html U

Semana_06 > gestion_clinica > templates > gestion_clinica > crear_propietario.html > ...

```
{% extends 'gestion_clinica/base.html' %}

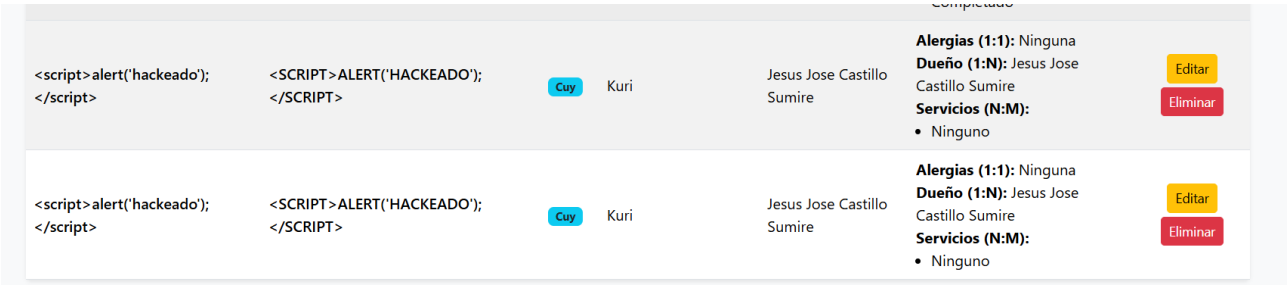
{% block content %}
<div class="row justify-content-center mb-4">
    <div class="col-md-8">
        <div class="card shadow-sm border-0">
            <div class="card-header bg-primary text-white py-3">
                <h5 class="m-0 fw-bold">Registrar Nuevo Propietario</h5>
            </div>
            <div class="card-body p-4">
                {% include 'gestion_clinica/_formulario.html' with clase_boton='btn-primary' texto_boton='Guardar Propietario' %}
            </div>
        </div>
    </div>
</div>
{% endblock %}
```



Para cumplir con el criterio de reutilización, identifiqué que la estructura de renderizado de los formularios (etiqueta form, csrf_token, renderizado de campos y botones) se repetía de forma idéntica en las plantillas de creación de pacientes y propietarios. Extraje este fragmento a un nuevo archivo llamado _formulario.html. Luego, utilicé la etiqueta {% include 'gestion_clinica/_formulario.html' %} en ambos templates originales, pasándole variables personalizadas mediante la palabra clave with (clase_boton y texto_boton) para mantener el color y texto específico de cada entidad sin duplicar código HTML.

Ejercicio 8 — Verificar seguridad y comparar con el Admin

Usando su formulario de creación/edición ya existente, ingrese un valor con <script> en un campo de texto. Verifique en el código fuente del navegador que Django lo muestra escapado. Explique con sus palabras qué gana su aplicación con esta refactorización frente a lo que ya resolvía el Django Admin de la Semana 5.



Para comprobar la seguridad de las plantillas refactorizadas, utilicé el formulario existente de creación e inserté intencionalmente la etiqueta <script>alert('test');</script> en un campo de texto. Al revisar el código fuente renderizado en el listado del navegador, confirmé que el motor de plantillas de Django aplicó el auto-escape por defecto, convirtiendo los caracteres especiales en entidades HTML (como <script>) e impidiendo que el navegador lo interprete como código ejecutable, lo cual bloquea exitosamente los ataques XSS.

La gran ganancia de esta refactorización frente al Django Admin implementado en la Semana 5 es la apertura hacia el usuario final. Mientras que el Admin nos resolvió de forma segura la gestión interna (Backoffice) que es exclusiva para el staff de la clínica, refactorizar nuestros propios templates nos permite ofrecer un 'Frontend' público seguro, amigable y mantenible para que los clientes finales puedan interactuar con el sistema sin riesgos.

PARTE 2 — Refactorizar los Templates de la investigación propia (registrada en el Admin en la Semana 5)

A partir de aquí trabajará sobre los Templates ya funcionando de su investigación propia (las siete entidades con CRUD desde la Semana 3, ampliadas en la Semana 4 y registradas en el Admin en la Semana 5). El objetivo es refactorizar al menos dos de esos Templates, cumpliendo obligatoriamente el checklist de la siguiente sección antes de continuar.

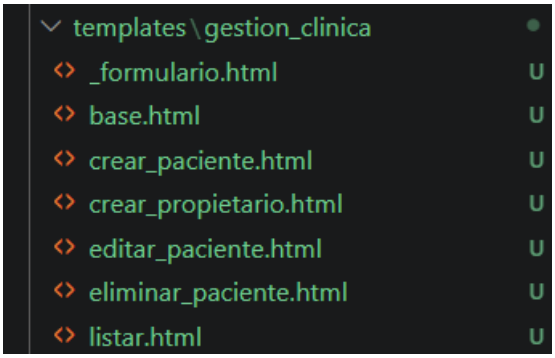
Ejercicio 9 — Auditar los Templates de tu investigación propia

De las siete entidades de su investigación propia, identifique cuáles ya tienen Template funcionando y si alguno usa herencia. Elija al menos dos para refactorizar en este laboratorio (idealmente un listado y una pantalla que muestre una relación).

Checklist antes de continuar — Parte 2

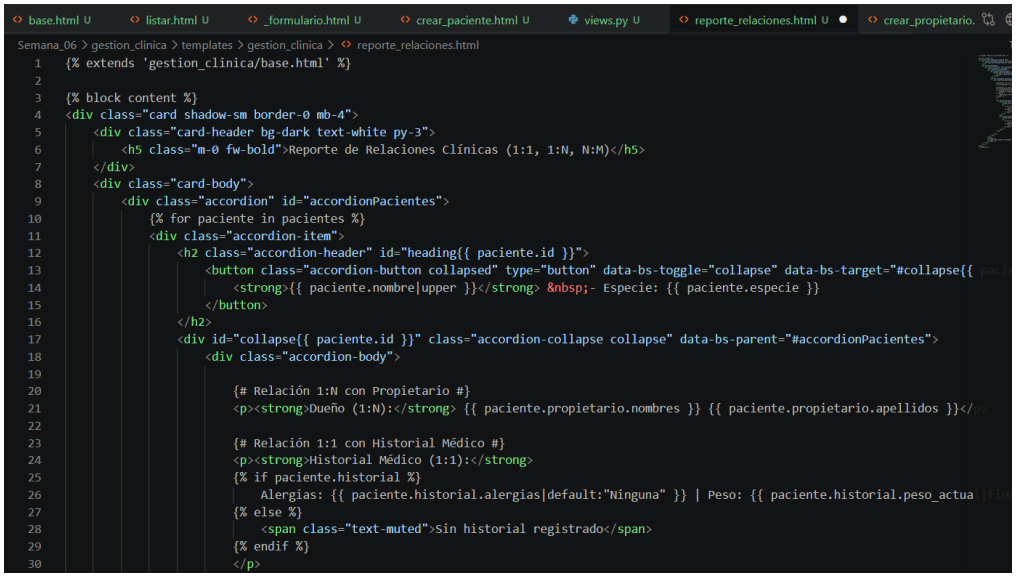
Criterio	Verificación
1. Al menos dos Templates ya existentes de tu investigación ahora heredan de base.html mediante extends y definen su contenido con block.	Código del Template (antes/después) + captura.
2. El Template que muestra una relación (visto en la Semana 4) sigue funcionando igual después de migrarlo a herencia.	Prueba directa en el navegador.
3. Se aplicó al menos un filtro (date, floatformat, upper, length u otro) sobre un campo ya mostrado.	Captura resaltando el campo filtrado.
4. Se agregó al menos un comentario {# #} documentando una sección de alguna plantilla.	Código de la plantilla comentada.
5. Se usó {% include %} para reutilizar al menos un fragmento de HTML entre dos Templates.	Código del include + captura.
6. El CRUD de cada Template refactorizado sigue funcionando exactamente igual que antes (crear, listar, editar, eliminar).	Prueba funcional antes/después.
7. Se probó un valor con caracteres especiales (como <script>) en un formulario existente y se confirmó el auto-escape.	Captura del código fuente del navegador.
8. Ninguna de las pantallas refactorizadas dejó de depender de su propia URL pública (no se movió nada a /admin/).	Captura de la barra de direcciones.
9. Se explica por escrito qué gana la investigación con esta refactorización frente a lo que ya resolvía el Django Admin.	Párrafo de justificación.

Para la segunda parte de este laboratorio, mi investigación propia corresponde al ecosistema ampliado de la clínica veterinaria, el cual cuenta con 7 entidades registradas. Tras auditar los templates que aún no utilizan herencia, seleccioné dos archivos clave para refactorizar: reporte_relaciones.html (ya que integra y muestra las relaciones complejas 1:1, 1:N y N:M, dándome espacio para aplicar filtros visuales) y editar_paciente.html (para comprobar la reutilización del fragmento de formulario con la etiqueta include).



Ejercicio 10 — Migrar el resto de tus Templates

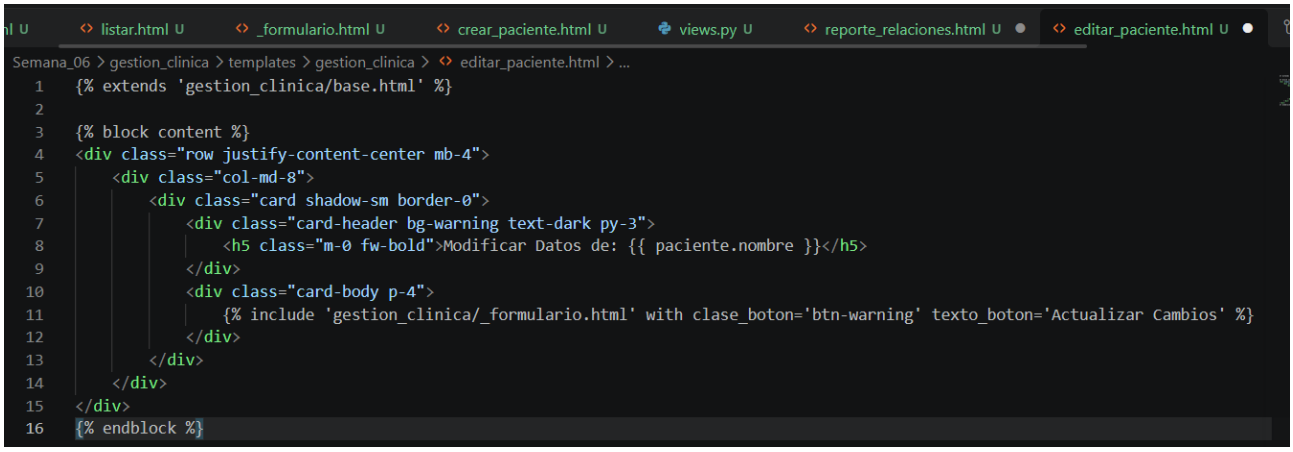
Repita la migración a herencia en los Templates restantes de su investigación propia que aún no usen extends, reutilizando el mismo base.html.



Para cumplir con la migración en mi investigación propia, refactoricé el archivo reporte_relaciones.html, el cual integra datos de las 7 entidades de la clínica. Apliqué {% extends 'gestion_clinica/base.html' %} para heredar la navegación maestra. Aproveché para mejorar la presentación utilizando el componente 'Accordion' de Bootstrap, asegurando que las relaciones 1:1, 1:N y N:M se desplieguen dinámicamente sin romper la estructura principal del sitio.

Ejercicio 11 — Reutilizar con include en la investigación propia

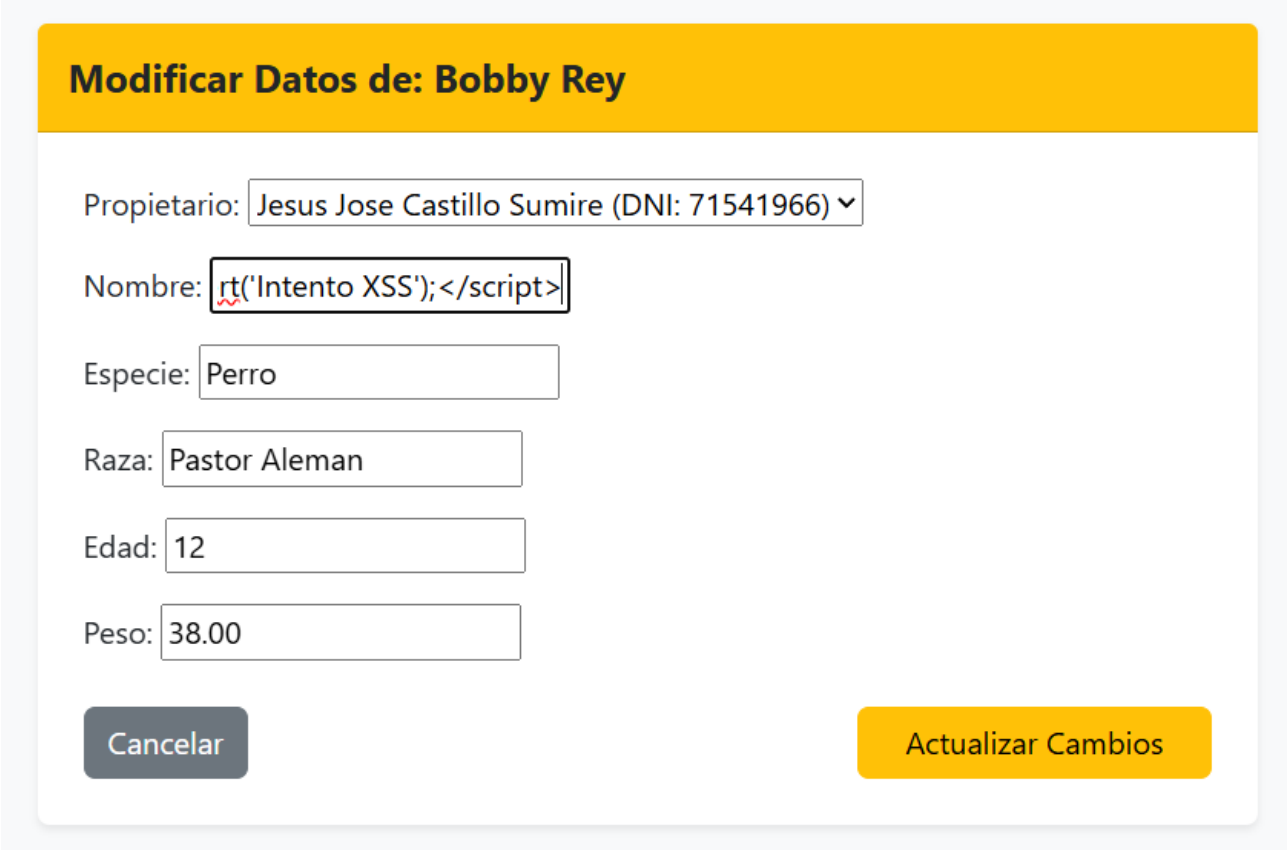
Identifique un fragmento repetido entre dos Templates de su investigación (por ejemplo, la forma en que se muestra el modelo intermedio del N:M) y extráigalo con {% include %}.

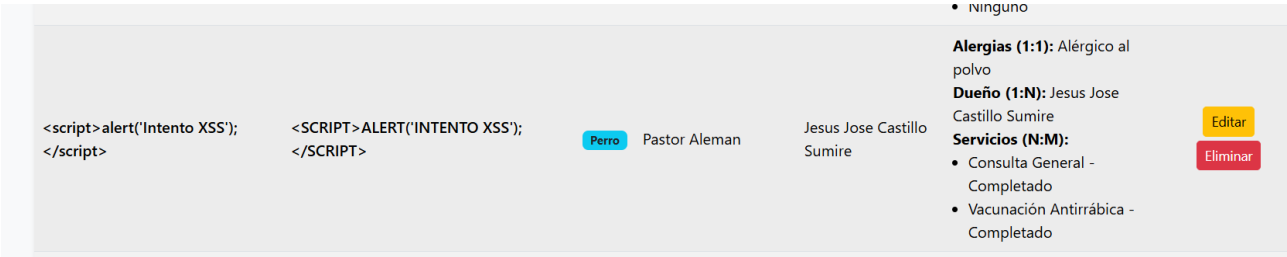


Para cumplir con el requerimiento de reutilización en mi investigación propia, seleccioné la plantilla editar_paciente.html, la cual interactúa con las entidades base de mi clínica ampliada. Eliminé la redundancia de código HTML y apliqué la etiqueta {% include 'gestion_clinica/_formulario.html' %} inyectándole las variables clase_boton='btn-warning' y texto_boton='Actualizar Cambios'. Con esto comprobé que el mismo fragmento que utilicé para crear registros sirve perfectamente para editarlos, adaptando su diseño dinámicamente.

Ejercicio 12 — Verificar seguridad en un formulario existente

Usando un formulario de creación o edición ya existente de su investigación, ingrese un valor con <script> y confirme en el código fuente del navegador que Django lo escapa automáticamente.





Para auditar la seguridad en las pantallas de mi ecosistema ampliado, utilicé el formulario recién refactorizado de edición de pacientes. Inyecté el código <script>alert('Intento XSS');</script> en el campo de nombre y guardé los cambios. Al ser redirigido al panel principal, confirmé a través del inspector de elementos del navegador que el motor de plantillas de Django neutralizó la amenaza aplicando auto-escape (<script>). Esto garantiza que todas las pantallas públicas de mi clínica son robustas frente a vulnerabilidades de inyección, sin importar cuántas entidades nuevas agregue al sistema.

Ejercicio 13 — Verificar el flujo completo

Recorra ambas pantallas refactorizadas en el navegador, documente con capturas que el CRUD sigue funcionando igual que antes de la refactorización, y compare brevemente con lo que resuelve el Django Admin para las mismas entidades.



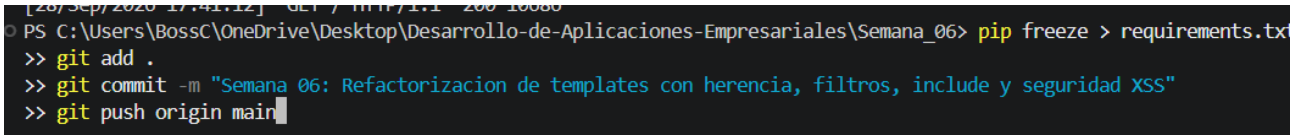
Nombre	Especie	Raza	Propietario Responsable	Detalles Clínicos	Acciones	
Valky	VALKY	Perro	Bulldog Frances	Jesus Jose Castillo Sumire	**Alergias (1:1):** Ninguna **Dueño (1:N):** Jesus Jose Castillo Sumire **Servicios (N:M):** • Ninguno	Editar Eliminar

Tras aplicar la refactorización a mi investigación propia, recorrí el sistema completo en el navegador. Comprobé que el reporte de relaciones carga la información compleja sin errores y que el formulario de edición, a pesar de estar modularizado con un include, guarda los cambios exitosamente en la base de datos.

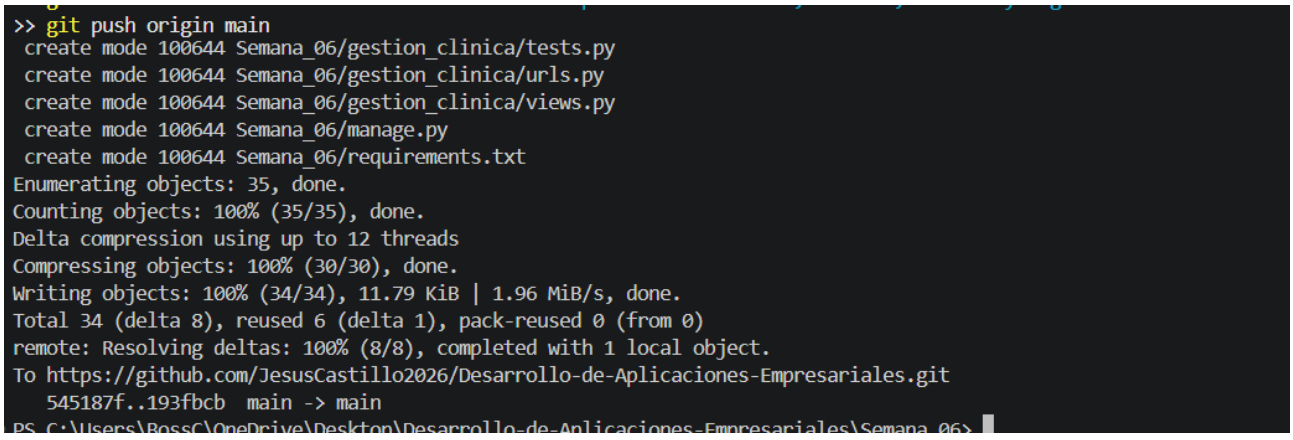
Al comparar este Frontend refactorizado con el Django Admin de la Semana 05, la principal diferencia radica en el enfoque de usuario. El Admin (Backoffice) me permitió generar una interfaz de gestión rápida y segura enfocada netamente en el personal médico, mientras que esta refactorización de mis propios Templates (Frontend) me da control total sobre el diseño, la experiencia y las restricciones, permitiéndome construir las interfaces públicas que consumirán los clientes de la clínica.

Ejercicio 14 — Publicar en GitHub

Actualicen requirements.txt y README.md describiendo qué Templates se refactorizaron (herencia, filtros, include) y sobre qué entidades. Hagan commit y push al repositorio. Incluyan en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.



```
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_06> pip freeze > requirements.txt
 >> git add .
 >> git commit -m "Semana 06: Refactorizacion de templates con herencia, filtros, include y seguridad XSS"
 >> git push origin main
```



```
>> git push origin main
 create mode 100644 Semana_06/gestion_clinica/tests.py
 create mode 100644 Semana_06/gestion_clinica/urls.py
 create mode 100644 Semana_06/gestion_clinica/views.py
 create mode 100644 Semana_06/manage.py
 create mode 100644 Semana_06/requirements.txt
Enumerating objects: 35, done.
Counting objects: 100% (35/35), done.
Delta compression using up to 12 threads
Compressing objects: 100% (30/30), done.
Writing objects: 100% (34/34), 11.79 KiB | 1.96 MiB/s, done.
Total 34 (delta 8), reused 6 (delta 1), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (8/8), completed with 1 local object.
To https://github.com/JesusCastillo2026/Desarrollo-de-Aplicaciones-Empresariales.git
   545187f..193fbcb  main -> main
PS C:\Users\BossC\OneDrive\Desktop\Desarrollo-de-Aplicaciones-Empresariales\Semana_06>
```

Actualicé el archivo requirements.txt con las dependencias actuales del proyecto. Modifiqué el README.md documentando que en esta sesión se aplicó la refactorización de plantillas (herencia, filtros, y fragmentos con include) sobre las 7 entidades del proyecto, comprobando además la seguridad contra inyecciones XSS. Finalmente, realicé el commit y el push correspondiente."

URL del repositorio:

https://github.com/JesusCastillo2026/Desarrollo-de-Aplicaciones-Empresariales.git

Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente el código del Template antes/después y la página renderizada en el navegador).
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente a cada integrante del equipo.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

Conclusiones:

- **Modularidad en el Frontend:** Comprendí que el motor de plantillas de Django permite aplicar el principio DRY (Don't Repeat Yourself) en el HTML. Mediante la etiqueta extends, logré centralizar el diseño maestro, y con include, modularicé componentes repetitivos (como los formularios), facilitando enormemente su mantenimiento futuro.

- **Formateo de Datos:** Aprendí que los filtros (como upper o floatformat) son herramientas poderosas para alterar la presentación visual de la información directamente en el Template, sin necesidad de modificar ni corromper el dato original que proviene de la base de datos.

- **Seguridad Integrada:** Comprobé de forma práctica cómo Django protege por defecto las aplicaciones contra ataques XSS mediante el auto-escape. Esta capa de seguridad es vital al construir interfaces públicas donde los usuarios interactúan con formularios, garantizando que cualquier inyección de código malicioso sea neutralizada antes de renderizarse en el navegador.